



Programação Estruturada vs. Orientada a Objetos

Paradigmas, História e a Evolução do Pensamento Computacional

1

Introdução aos Paradigmas

2

Análise da Programação Estruturada

3

Análise da Programação Orientada a Objetos (POO)

4

Comparativo Técnico e Prático

Além da Sintaxe: A Importância dos Paradigmas

A Lição Principal

Programar não é apenas traduzir lógica, é sobre *como pensar*.

O que são Paradigmas?

"Lentes" ou modelos mentais desenvolvidos pela ciência da computação para resolver problemas.

Objetivo

Entender a diferença é crucial para escolher a ferramenta certa e compreender a evolução do software moderno.

O Grande Embate

Estruturada

Foco no fluxo e ações.

Orientada a Objetos

Foco nas entidades e interações.





Programação Estruturada: Lógica Linear e Ações

Contexto Histórico

Décadas de 1960/70. Figuras como Edsger W. Dijkstra e o fim do "Espaguete Code" (uso excessivo de GOTO).

Princípio Central: Top-Down

De cima para baixo. Divide-se o problema principal em subproblemas (funções/procedimentos).

Separação Rígida

Dados: Passivos (apenas existem na memória).

Comportamento: Ativo (funções que manipulam os dados).

Foco

Nos "verbos" (ações) e no fluxo de controle.



O Mundo como uma Receita de Bolo

Cenário

Programar um sistema para fazer um bolo.

Abordagem

- Função principal:
`fazerBolo()`.
- Sub-rotinas:
`baterIngredientes()`,
`assar()`, `deixarEsfriar()`.

Fluxo de Dados

- Os ingredientes (dados) são passados como parâmetros de uma função para outra.
- A função `assar()` não se importa com a origem, apenas recebe a massa e aplica calor.

Resumo

Uma sequência lógica de instruções passo a passo.

Vantagens e Limitações da Abordagem Linear

✓ Vantagens

- Simplicidade e previsibilidade.
- Ideal para problemas pequenos, scripts, ETL e algoritmos matemáticos.

✗ Desvantagens

- **Manutenção:** Torna-se um pesadelo em sistemas grandes.
- **Efeito Cascata:** Alterar a estrutura de um dado exige reescrever dezenas de funções.
- **Risco:** Dados globais ou compartilhados facilitam bugs inesperados.



POO: A Arquitetura de Entidades

Contexto

Resposta à complexidade dos sistemas nos anos 80/90.

Mudança Radical

Foco nos **Substantivos** (Entidades).

Princípio Central: Encapsulamento

Dados e Comportamento vivem juntos dentro do Objeto.

Abordagem Bottom-Up

O sistema é uma comunidade de objetos que interagem via mensagens.



O Mundo como um Ecosystema de Entidades

Cenário

O mesmo problema da comida, agora em um restaurante.

Objetos

Garçom, Cozinheiro, Cliente, Prato.

Interação

- O Cliente não vai à cozinha (violação de encapsulamento).
- O Cliente envia uma mensagem (pedido) ao Garçom.

Responsabilidade

Cada objeto sabe fazer seu trabalho internamente, escondendo os detalhes de implementação dos outros.

Escalabilidade e Manutenção

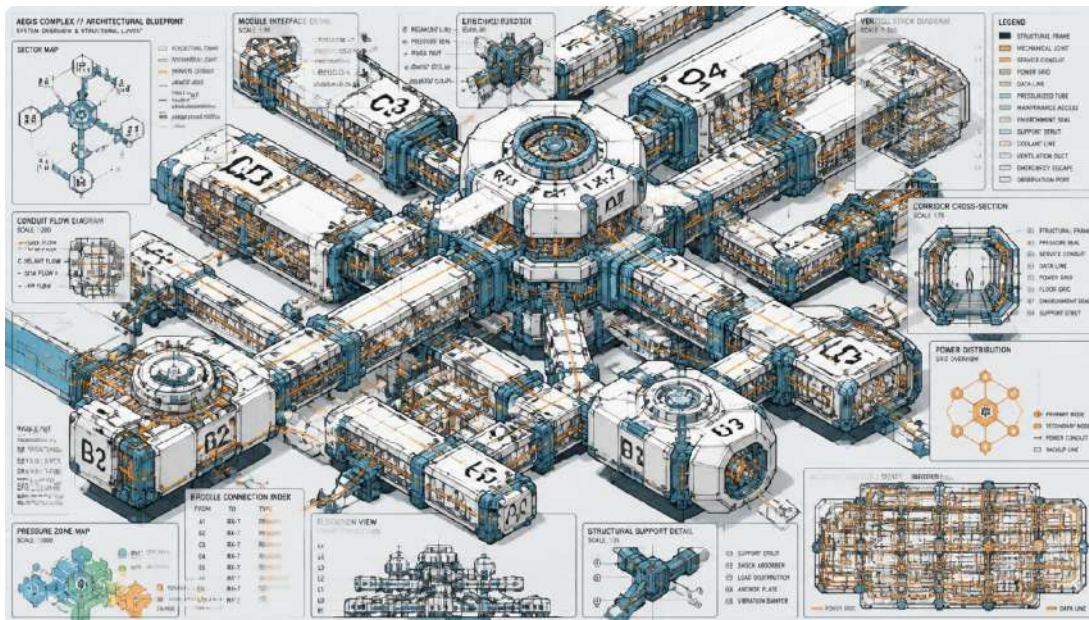
✓ Vantagens

- **Escalabilidade:** Construção de sistemas gigantescos.
- **Manutenção:** Alterações internas não quebram o sistema (graças ao encapsulamento).
- **Reutilização:** Herança e Composição permitem estender comportamentos facilmente.



Foco

Qualidade na arquitetura e organização do domínio de negócio.



Divergências: Dados e Reutilização

1. Segurança e Manipulação de Dados

Estruturada: Dados expostos. Difícil rastrear quem alterou o quê (bug hunting difícil).

POO: Dados privados (protegidos). O objeto é o guardião do seu estado, garantindo regras de negócio.

2. Reutilização de Código

Estruturada: Bibliotecas de funções (copiar/colar ou importar). Sem extensão nativa.

POO: Estrutural. Herança e Composição permitem criar especializações sem reescrever a base.

Modelagem, Equipes e Escolha do Paradigma

3. Modelagem do Mundo Real

Estruturada: Fluxo de processos (ex: pipeline de vídeo).

POO: Entidades interagindo (ex: sistema bancário complexo).

4. Trabalho em Equipe

Estruturada: Conflitos em blocos globais de funções.

POO: Módulos isolados permitem trabalho paralelo com mínima interferência.

 **Conclusão:** Não existe "bala de prata". A escolha depende da complexidade do domínio e da escala do projeto.